Análisis Algorítmico y de Estructuras

1. Análisis de Collections y Estructuras (QuindioExpress.java)

- **Paquetes Pendientes (LinkedHashMap<String, Paquete>)**

  ¿Cuánto cuesta?
  La complejidad asintótica para operaciones de inserción y búsqueda por código es de O(1) en el caso promedio.

  Justificación
  Se implementó un LinkedHashMap en lugar de un HashMap estándar debido a que el primero preserva el orden de inserción original de los elementos. Para satisfacer el requerimiento de negocio referente al despacho de paquetes y orden de llegada, permitiendo búsquedas instantáneas simultáneamente.

  Alternativa descartada
  El uso de ArrayList habría degradado el rendimiento del sistema, elevando la complejidad a O(N) para la búsqueda de un paquete por su código o para su eliminación posterior al despacho.

- **Municipios de Destino (TreeSet<String>)**

  ¿Cuánto cuesta?
  Las operaciones de inserción y búsqueda presentan una complejidad logarítmica de O(log N).

  Justificación
  El problema exige listar los municipios sin permitir registros duplicados. Al heredar de la interfaz Set, es por esto que se seleccionó TreeSet sobre HashSet dado que garantiza un ordenamiento natural (Alfabetico) de los elementos, facilitando la generación directa de reportes sin requerir algoritmos de clasificación adicionales.

- **Despacho Prioritario**

  ¿Cuánto cuesta?
  La inserción y la extracción del elemento de mayor prioridad, operan en O(log N). La consulta del elemento en el tope de la cola (peek) es de O(1).

Justificación
Constituye la estructura de datos optima para gestionar despachos basados en urgencia. Se implementó mediante la inyección de un Comparator en cascada que evalua, en orden de precedencia: mayor prioridad, menor tiempo estimado y orden del código. Esto asegura que la extracción siempre retorne el paquete idóneo de forma eficiente, sin necesidad de reordenar o recorrer listas secuenciales (O(N log N)) en cada operación de despacho.

- **Historial de Entregas (Lista Simplemente Enlazada Propia)**

¿Cuánto cuesta?
La inserción opera en O(N) dado que los nuevos registros se añaden al final para preservar la secuencia cronológica. Las operaciones de obtención y eliminación también operan en O(N) en el peor de los escenarios.

Justificación
Su implementación responde al cumplimiento estricto de las restricciones que prohíben el uso de Colecciones nativas de Java para este módulo. El diseño basado en tipos genéricos (<T>) y la implementación de las interfaces Iterable e Iterator garantizan el encapsulamiento de los nodos en memoria, permitiendo el recorrido seguro de la estructura sin exponer referencias internas o punteros.

2. **Análisis de Algoritmos (Clase AnalisisDatos.java)**

- **Búsqueda Binaria (buscarPaqueteBinario)**

¿Cuánto cuesta?
Presenta una complejidad temporal logarítmica de O(log N) y una complejidad espacial de O (1) al implementarse de manera iterativa.

Justificación
Al segmentar el espacio de búsqueda por la mitad en cada iteración, ofrece una eficiencia computacional superior frente a una búsqueda secuencial o lineal convencional (O(N)).

- **Estrategia Divide y Vencerás (aplicarDivideYVenceras – Paquete de Mayor Peso)**

¿Cuánto cuesta?
La complejidad temporal es de O(N), ya que el algoritmo debe evaluar cada elemento del arreglo al menos una vez para determinar el peso máximo. La complejidad espacial,

derivada de la profundidad de la pila de llamadas recursivas, es de O(log N) al particionar sistemáticamente el arreglo en mitades.

Justificación
En contraposición a un recorrido iterativo con variable acumuladora, este enfoque segmenta el problema original en sub-arreglos hasta alcanzar el caso base (un único elemento). En la etapa de combinación, el algoritmo determina el peso máximo evaluando y comparando los retornos de los subproblemas izquierdo y derecho.

Alternativa Descartada
Un recorrido secuencial mediante un ciclo for ofrecería la misma complejidad temporal (O(N)) optimizando la memoria (O(1)), No obstante, dicho enfoque fue descartado para dar cumplimiento a las rubricas de evaluación que exigen evidenciar las fases algorítmicas de partición y combinación.